

PHP Parser

coverage 93%

This is a PHP parser written in PHP. Its purpose is to simplify static code analysis and manipulation.

Documentation for version 5.x (current; for running on PHP ≥ 7.4 ; for parsing PHP 7.0 to PHP 8.4, with limited support for parsing PHP 5.x).

Documentation for version 4.x (supported; for running on PHP ≥ 7.0 ; for parsing PHP 5.2 to PHP 8.3).

Features

The main features provided by this library are:

- Parsing PHP 7, and PHP 8 code into an abstract syntax tree (AST).
 - Invalid code can be parsed into a partial AST.
 - The AST contains accurate location information.
- Dumping the AST in human-readable form.
- Converting an AST back to PHP code.
 - Formatting can be preserved for partially changed ASTs.
- Infrastructure to traverse and modify ASTs.
- Resolution of namespaced names.
- Evaluation of constant expressions.
- Builders to simplify AST construction for code generation.
- Converting an AST into JSON and back.

Quick Start

Install the library using composer:

```
php composer.phar require nikic/php-parser
```

Parse some PHP code into an AST and dump the result in human-readable form:

```
<?php
use PhpParser\Error;
use PhpParser\NodeDumper;
use PhpParser\ParserFactory;

$code = <<<'CODE'
<?php

function test($foo)
{
    var_dump($foo);
```

```

}
CODE;

$parser = (new ParserFactory())->createForNewestSupportedVersion();
try {
    $ast = $parser->parse($code);
} catch (Error $error) {
    echo "Parse error: {$error->getMessage()}\n";
    return;
}

$dumper = new NodeDumper;
echo $dumper->dump($ast) . "\n";

```

This dumps an AST looking something like this:

```

array(
  0: Stmt_Function(
    attrGroups: array(
    )
    byRef: false
    name: Identifier(
      name: test
    )
    params: array(
      0: Param(
        attrGroups: array(
        )
        flags: 0
        type: null
        byRef: false
        variadic: false
        var: Expr_Variable(
          name: foo
        )
        default: null
      )
    )
    returnType: null
    stmts: array(
      0: Stmt_Expression(
        expr: Expr_FuncCall(
          name: Name(
            name: var_dump
          )
          args: array(
            0: Arg(

```

```
name: null  
value: Expr_Variable(  
    name: foo  
)  
byRef: false  
unpack: false  
  
)  
  
)  
  
)  
  
)  
  
)
```

Let's traverse the AST and perform some kind of modification. For example, drop all function bodies:

```
use PhpParser\Node;
use PhpParser\Node\Stmt\Function_;
use PhpParser\NodeTraverser;
use PhpParser\NodeVisitorAbstract;

$traverser = new NodeTraverser();
$traverser->addVisitor(new class extends NodeVisitorAbstract {
    public function enterNode(Node $node) {
        if ($node instanceof Function_) {
            // Clean out the function body
            $node->stmts = [];
        }
    }
});

$ast = $traverser->traverse($ast);
echo $dumper->dump($ast) . "\n";
```

This gives us an AST where the `Function_::$stmts` are empty:

```
array(  
  0: Stmt_Function(  
    attrGroups: array(  
      )  
    byRef: false  
    name: Identifier(  
      name: test  
    )  
    params: array(  
      0: Param(  
        attrGroups: array(  
          )
```

```

        )
        type: null
        byRef: false
        variadic: false
        var: Expr_Variable(
            name: foo
        )
        default: null
    )
)
returnType: null
stmts: array(
)
)
)

```

Finally, we can convert the new AST back to PHP code:

```

use PhpParser\PrettyPrinter;

$prettyPrinter = new PrettyPrinter\Standard;
echo $prettyPrinter->prettyPrintFile($ast);

```

This gives us our original code, minus the `var_dump()` call inside the function:

```

<?php

function test($foo)
{
}

```

For a more comprehensive introduction, see the documentation.

Documentation

1. Introduction
2. Usage of basic components

Component documentation:

- Walking the AST
 - Node visitors
 - Modifying the AST from a visitor
 - Short-circuiting traversals
 - Interleaved visitors
 - Simple node finding API
 - Parent and sibling references
- Name resolution
 - Name resolver options

- Name resolution context
- Pretty printing
 - Converting AST back to PHP code
 - Customizing formatting
 - Formatting-preserving code transformations
- AST builders
 - Fluent builders for AST nodes
- Lexer
 - Emulation
 - Tokens, positions and attributes
- Error handling
 - Column information for errors
 - Error recovery (parsing of syntactically incorrect code)
- Constant expression evaluation
 - Evaluating constant/property/etc initializers
 - Handling errors and unsupported expressions
- JSON representation
 - JSON encoding and decoding of ASTs
- Performance
 - Disabling Xdebug
 - Reusing objects
 - Garbage collection impact
- Frequently asked questions
 - Parent and sibling references